

Curvas

Curvas

Curvas são utilizadas para

- Desenho de tanto de curvas
- Modelagem de superfícies;
- especificação de caminhos;
- movimentos de animações e de câmeras;

As curvas mais conhecidas são:

- Bézier;
- Hermite;
- Splines.

Tipos de curvas

Interpolação:

A curva passa sobre os pontos de controle;

Usado em digitalização;



Aproximação:

A curva aproxima os pontos de controle;

Usando na modelagem de objetos.



Representação de curvas

Curvas são representadas por partes (segmentos de curva) através de **polinômios de grau baixo**.

Algumas curvas não podem ser facilmente descritas por expressões analíticas em toda sua extensão. Assim, utilizam-se **conjuntos** de (segmentos de) **curvas menores**, unidas pelas extremidades.

Geralmente são utilizados **polinômios de grau 3** para a descrição de uma curva:

- Quanto **maior** for o **grau** das funções, **mais complexos** são os **cálculos**, além de surgirem outros problemas como instabilidade numérica;
- Polinômios de **grau 3** são **flexíveis** e suprem a maioria dos requisitos de aplicações práticas;
- Eles também representam **espaço tridimensional**, ou seja, não são planares.

Representação de curvas

Cada segmento de curva é definido por um conjunto de pontos discretos (**pontos de controle**), juntamente com um conjunto de funções básicas.

Para funções (polinômios) de grau 3, utilizam-se 4 pontos de controle (P_0 , P_1 , P_2 e P_3), e 4 funções bases com representação paramétrica:

- $x = x(t);$
- $y = y(t);$
- $P(t) = (x(t), y(t))$

Para 3D, basta adicionar mais uma coordenada:

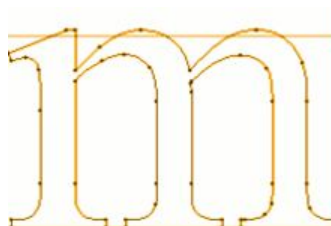
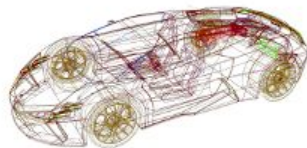
- $P(t) = (x(t), y(t), z(t));$

Os extremos e o comprimento da curva são fixos pelo intervalo de variação do parâmetro, frequentemente normalizado para $0 \leq t \leq 1$.

Curvas de Bézier

Curvas de Bézier são curvas fáceis de se utilizar e capazes de descrever diversas formas;

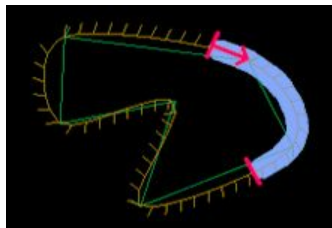
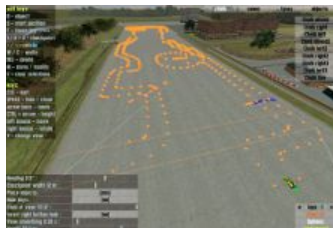
Elas são comumente utilizadas na representação de **fontes de texto**, bem como em formas no **design de veículos**;



Curvas de Bézier

Em jogos, curvas de Bézier são bastante populares, pois sua descrição matemática são **compactas, intuitivas e elegantes**, além de serem fáceis de computar tanto em duas quanto em três dimensões;

Curvas de Bézier também podem ser utilizadas para **descrição de rotas**, como por exemplo, uma linha de corrida de um jogo de corrida.



Descrição matemática

Uma curva de Bézier é descrita matematicamente como uma **função** que recebe um parâmetro **t**.

O valor da função é um ponto da curva, que depende do parâmetro **t**, bem como de um conjunto de pontos denominados **pontos de controle**.

O **primeiro** e o **último** ponto de controle representam, respectivamente, o **ponto inicial e final da curva**.

Em geral, a **curva não passa sobre os outros pontos de controle**.

O valor de **t** varia entre 0 e 1, sendo **0** correspondente ao **ponto inicial** da curva e **1** ao **ponto final**. Valores de **t** entre 0 e 1 correspondem a outros pontos na curva.

Descrição matemática

O exemplo abaixo é o tipo de “curva” de Bézier mais simples que existe, que possui somente 2 pontos de controle e representa **um segmento de reta**:

- $[x, y] = (1 - t)P_0 + tP_1$

Os pontos P0 e P1 são os pontos de controle:

- Substituindo na equação $t=0$, $[x, y]$ passa a valer P0
- Ao substituir por $t=1$, $[x, y]$ passa a valer P1



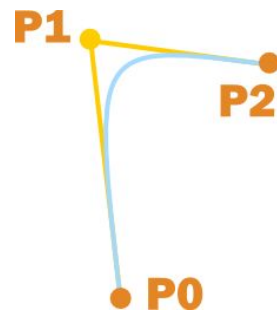
Descrição matemática

Para obtermos formas mais interessantes, nós precisamos trabalhar com **mais pontos de controle**, sendo que a quantidade de pontos de controle de uma curva determina o grau da curva:

- Curvas com **2 pontos de controle**, ou **curvas lineares**, são denominadas curvas de **grau 1**;
- Curvas com **3 pontos de controle**, ou **curvas quadráticas**, são curvas de **grau 2**;

A fórmula para uma curva quadrática é:

- $[x, y] = (1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2$



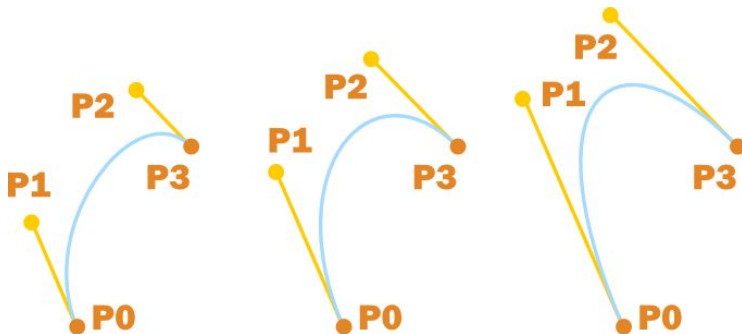
Curvas cúbicas

Curvas cúbicas (ou de grau 3) são as mais frequentemente utilizadas, portanto serão as que daremos mais foco.

Essas requerem 4 pontos de controle. A fórmula para curvas cúbicas de Bézier é:

- $[x, y] = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3$

Difícilmente precisaremos trabalhar com curvas de um maior grau.



Implementação

Para implementarmos uma curva de Bézier, precisamos definir:

- O **grau** da curva
- Os **pontos de controle**
- A **precisão** da mesma

A **precisão** define de quanto em quanto o valor de **t** irá variar entre 0 e 1.

- Quanto menor o “pulo”, mais precisa será a curva.
- Uma curva com **t** pulando de 0.1 em 0.1 será mais precisa que uma curva com **t** pulando de 0.2 em 0.2, pois a primeira terá mais pontos.

Implementação

Por exemplo, se vamos desenhar uma curva com **t** variando de 0.1 em 0.1, calcularemos o valor da função (pontos da curva) para os seguintes valores de **t**:

- $t = 0.0$
- $t = 0.1$
- $t = 0.2$
- $t = 0.3$
- $t = 0.4$
- $t = 0.5$
- $t = 0.6$
- $t = 0.7$
- $t = 0.8$
- $t = 0.9$
- $t = 1.0$

Implementação

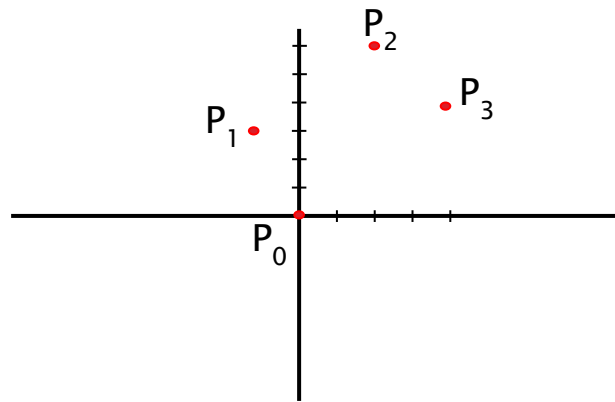
Já se decidirmos desenhar uma curva com t variando de 0.2 em 0.2, calcularemos os pontos da curva para os seguintes valores de t :

- $t = 0.0$
- $t = 0.2$
- $t = 0.4$
- $t = 0.6$
- $t = 0.8$
- $t = 1.0$

Implementação

Neste exemplo, como ficaria uma **curva cúbica**, com **t** variando de 0.2 em 0.2, cujos pontos controle são:

- $P_0 = \{0, 0\}$
- $P_1 = \{-1, 3\}$
- $P_2 = \{2, 6\}$
- $P_3 = \{4, 4\}$



Implementação

Calculamos, para cada valor de t , o ponto correspondente a tal segmento:

$t=0.00$:

- $[x,y] = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$
- $[x,y] = (1-0)^3 \{0,0\} + 3(1-0)^2 \cdot 0 \{-1,3\} + 3(1-0) 0^2 \{2,6\} + 0^3 \{4,4\}$
- $[x,y] = \{0,0\} + \{0,0\} + \{0,0\} + \{0,0\}$
- $[x,y] = \{0,0\}$

$t=0.20$:

- $[x,y] = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$
- $[x,y] = (1-0.20)^3 \{0,0\} + 3(1-0.20)^2 \cdot 0.20 \{-1,3\} + 3(1-0.20) 0.20^2 \{2,6\} + 0.20^3 \{4,4\}$
- $[x,y] = 0.80^3 \{0,0\} + 3 \cdot 0.80^2 \cdot 0.20 \{-1,3\} + 3 \cdot 0.80 \cdot 0.04 \{2,6\} + 0.008 \{4,4\}$
- $[x,y] = 0.512 \{0,0\} + 0.384 \{-1,3\} + 0.096 \{2,6\} + 0.008 \{4,4\}$
- $[x,y] = \{0,0\} + \{-0.384, 1.152\} + \{0.192, 0.576\} + \{0.032, 0.032\}$
- $[x,y] = \{-0.16, 1.76\}$

Implementação

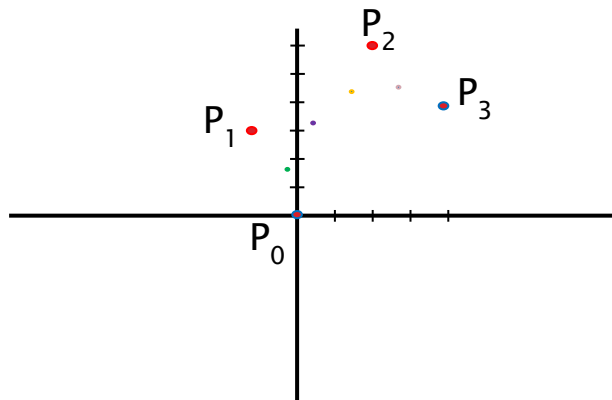
e seguindo a mesma ideia...

- $t=0.40: \{0.40, 3.28\}$
- $t=0.60: \{1.44, 4.32\}$
- $t=0.80: \{2.72, 4.64\}$
- $t=1.00: \{4.00, 4.00\}$

Implementação

Resultado:

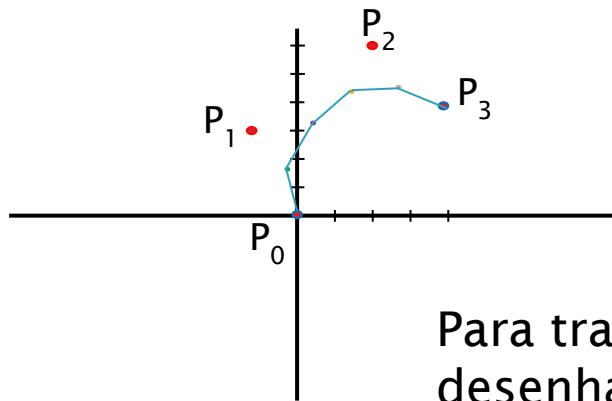
- $t=0.00: \{0.00, 0.00\}$
- $t=0.20: \{-0.16, 1.76\}$
- $t=0.40: \{0.40, 3.28\}$
- $t=0.60: \{1.44, 4.32\}$
- $t=0.80: \{2.72, 4.64\}$
- $t=1.00: \{4.00, 4.00\}$



Implementação

Resultado:

- $t=0.00: \{0.00, 0.00\}$
- $t=0.20: \{-0.16, 1.76\}$
- $t=0.40: \{0.40, 3.28\}$
- $t=0.60: \{1.44, 4.32\}$
- $t=0.80: \{2.72, 4.64\}$
- $t=1.00: \{4.00, 4.00\}$



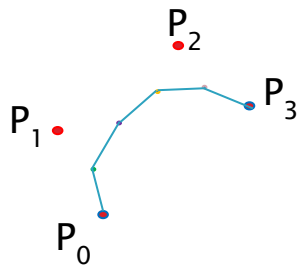
Para traçar a curva, podemos desenhar retas entre um ponto e outro.

Obviamente, quanto mais segmentos, maior a qualidade da curva!

Implementação

Resultado:

- $t=0.00: \{0.00, 0.00\}$
- $t=0.20: \{-0.16, 1.76\}$
- $t=0.40: \{0.40, 3.28\}$
- $t=0.60: \{1.44, 4.32\}$
- $t=0.80: \{2.72, 4.64\}$
- $t=1.00: \{4.00, 4.00\}$



Para traçar a curva, podemos desenhar retas entre um ponto e outro.

Obviamente, quanto mais segmentos, maior a qualidade da curva!

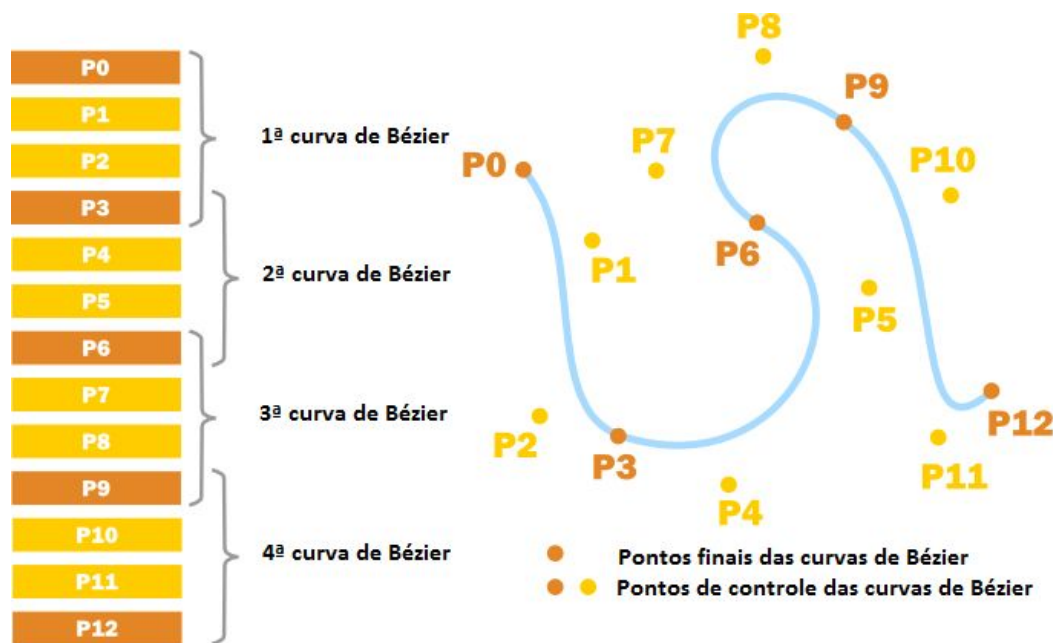
Curvas complexas/Bézier Path

Se quisermos representar uma curva mais complexa, podemos utilizar:

- Uma **única curva** de Bézier, com um **grau maior**;
- Uma **combinação de curvas de Bézier com um grau menor**, que geralmente é mais fácil de implementar e mais eficiente em relação a processamento.

Curvas complexas/Bézier Path

O exemplo abaixo mostra como poderia ser feita a implementação de uma curva complexa através da combinação de várias curvas cúbicas:



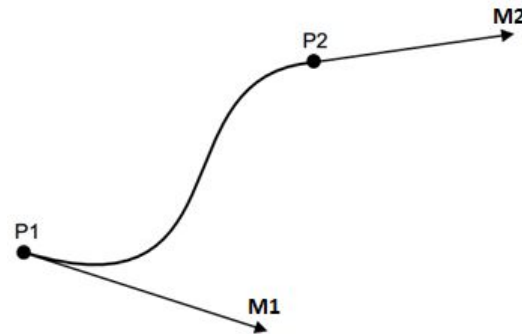
Curvas de Hermite

Curvas de Hermite foram uma das **primeiras representações matemáticas** de curvas complexas;

Hermite definiu uma curva utilizando uma **equação polinomial**, cuja curva é definida a partir de **dois pontos** e **dois vetores tangentes** que determinam sua forma;

Para gerar uma curva de Hermite, são necessários, portanto, 4 fatores:

- P1: Ponto inicial da curva;
- P2: Ponto final da curva;
- M1: Vetor tangente e peso na curva, indica como a curva sai de P1;
- M2: Vetor tangente e peso na curva, indica como a curva chega em P2;



Curvas de Hermite

A partir dos valores de P1, P2, M1 e M2, podemos definir os pontos de uma curva de Hermite a partir da equação abaixo:

$$H(t) = (2t^3 - 3t^2 + 1)P1 + (t^3 - 2t^2 + t)M1 + (-2t^3 + 3t^2)P2 + (t^3 - t^2)M2$$

Com t variando de 0 a 1, sendo que:

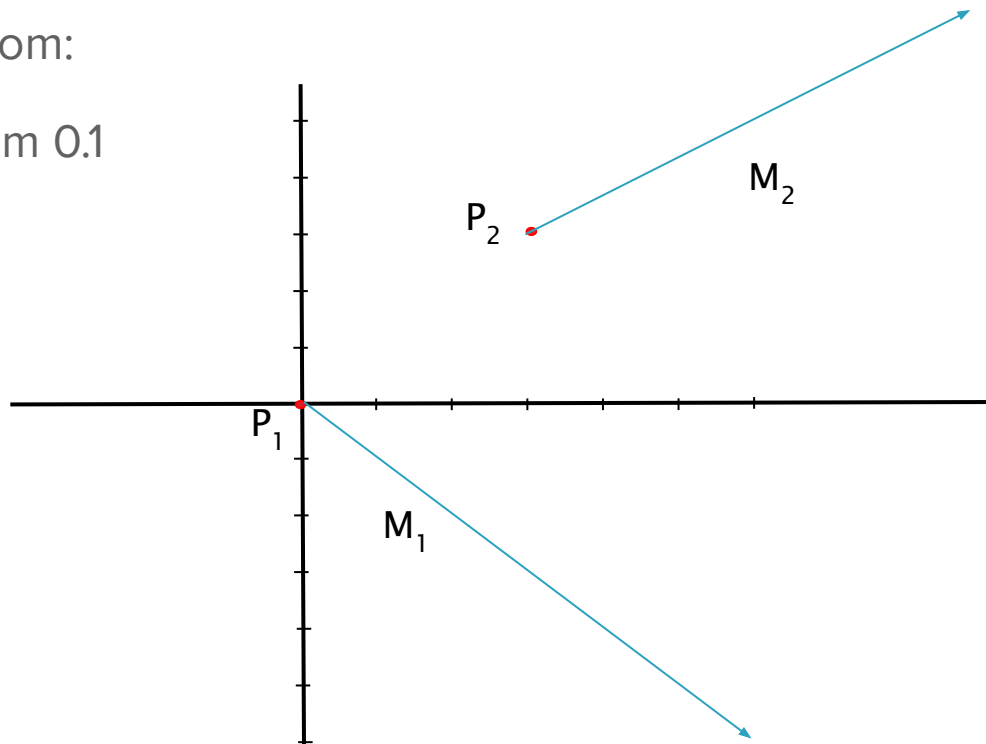
- t=0: ponto inicial da curva P1;
- t=1: ponto final da curva P2;

Novamente, quanto maior a quantidade de segmentos, mais precisa será a curva.

Exemplo

Desenhar uma curva de 10 segmentos com:

- t variando de 0 a 1, pulando de 0.1 em 0.1
- $P_1 = \{0, 0\}$
- $P_2 = \{3, 3\}$
- $M_1 = \{6, -6\}$
- $M_2 = \{6, 4\}$

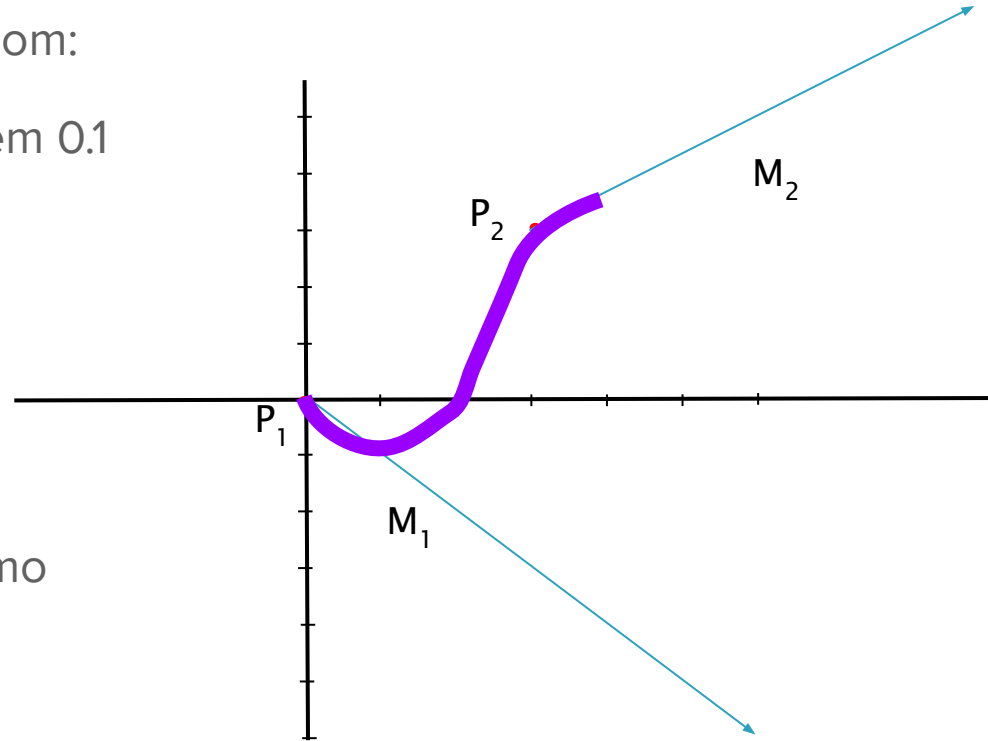


Exemplo

Desenhar uma curva de 10 segmentos com:

- t variando de 0 a 1, pulando de 0.1 em 0.1
- $P_1 = \{0, 0\}$
- $P_2 = \{3, 3\}$
- $M_1 = \{6, -6\}$
- $M_2 = \{6, 4\}$

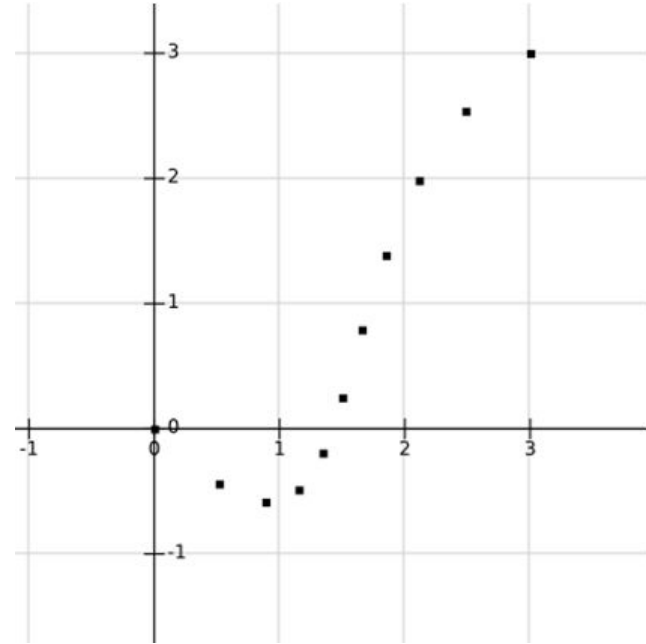
O resultado esperado deveria ser próximo a isso...



Exemplo

Resultado:

- $t=0.00$, $H(0.00)=\{0.000, 0.000\}$
- $t=0.10$, $H(0.10)=\{0.516, -0.438\}$
- $t=0.20$, $H(0.20)=\{0.888, -0.584\}$
- $t=0.30$, $H(0.30)=\{1.152, -0.486\}$
- $t=0.40$, $H(0.40)=\{1.344, -0.192\}$
- $t=0.50$, $H(0.50)=\{1.500, 0.250\}$
- $t=0.60$, $H(0.60)=\{1.656, 0.792\}$
- $t=0.70$, $H(0.70)=\{1.848, 1.386\}$
- $t=0.80$, $H(0.80)=\{2.112, 1.984\}$
- $t=0.90$, $H(0.90)=\{2.484, 2.538\}$
- $t=1.00$, $H(1.00)=\{3.000, 3.000\}$



Exemplo

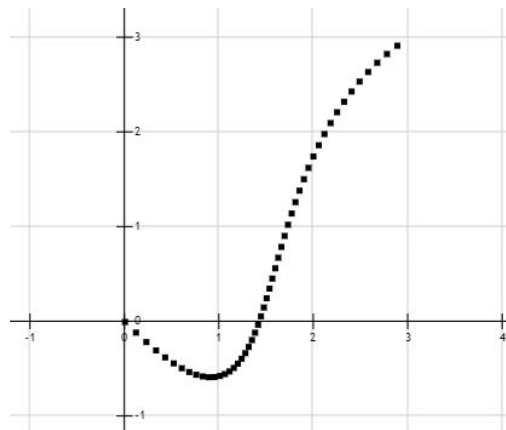
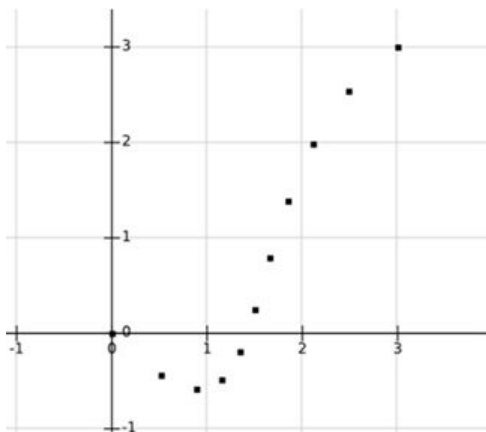
Resultado:

- $t=0.00$, $H(0.00)=\{0.000, 0.000\}$
- $t=0.10$, $H(0.10)=\{0.516, -0.438\}$
- $t=0.20$, $H(0.20)=\{0.888, -0.584\}$
- $t=0.30$, $H(0.30)=\{1.152, -0.486\}$
- $t=0.40$, $H(0.40)=\{1.344, -0.192\}$
- $t=0.50$, $H(0.50)=\{1.500, 0.250\}$
- $t=0.60$, $H(0.60)=\{1.656, 0.792\}$
- $t=0.70$, $H(0.70)=\{1.848, 1.386\}$
- $t=0.80$, $H(0.80)=\{2.112, 1.984\}$
- $t=0.90$, $H(0.90)=\{2.484, 2.538\}$
- $t=1.00$, $H(1.00)=\{3.000, 3.000\}$



Exemplo

A direita, a curva com 10 segmentos, e a esquerda a mesma curva, mas com 50 segmentos (t variando de 0 a 1, pulando de 0.02 em 0.02):



Direção da Curva em um Determinado Ponto

Em muitas aplicações gráficas, saber apenas a posição (x, y, z) em um ponto da curva não basta. Precisamos da orientação (vetor tangente) para:

- Alinhar objetos ao longo de um caminho.
- Calcular a orientação de superfícies para iluminação (normais).
- Gerar movimento suave (câmeras ou personagens).

O Objetivo: Obter um vetor que represente a direção do deslocamento da curva naquele instante exato.

A Intuição da Derivada

Conceito: A derivada mede a "taxa de variação" de uma função.

Geometria: Se tomarmos dois pontos da curva (t e $t + \Delta t$) e calcularmos a diferença de posição, temos uma **secante**. Quando o intervalo (Δt) se aproxima de zero, a secante se torna a reta tangente.

Cálculo Simplificado (Regra da Potência): Se temos um termo do tipo $f(t) = a \cdot t^n$, sua derivada é $f'(t) = n \cdot a \cdot t_{n-1}$.

Exemplos:

- A derivada de t^2 é $2t$.
- A derivada de t^3 é $3t^2$.

Constantes: A derivada de uma constante (como um ponto de controle P0) é zero, pois ele não muda com t .

Derivada como Vetor de Movimento

$P(t)$ representa uma posição ao longo da curva. Se avançarmos um pouco no tempo:

- $P(t + \Delta t) \rightarrow$ nova posição

A diferença entre os dois pontos $P(t + \Delta t) - P(t)$ indica a **direção do movimento** e o **sentido da curva naquele ponto**.

A **derivada** é um **vetor** que **aponta para onde a curva está indo**

Derivada de uma Curva de Bézier Cúbica

Definição Paramétrica: Uma curva de Bézier cúbica é definida por quatro pontos de controle (P0, P1, P2, P3):

- $$B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3$$

A Derivada (Vetor Tangente): Para encontrar a direção em qualquer t , derivamos a função em relação a t :

- $$B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2)$$

Interpretação Geométrica: O vetor resultante indica a direção e a velocidade (magnitude) da curva no ponto t .

Derivada de uma Curva de Bézier Cúbica

$$B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2)$$

Nota: Ao derivar a curva, os termos deixam de representar posições absolutas e passam a representar variações entre pontos consecutivos.

Por isso, surgem diferenças como $(P_1 - P_0)$, $(P_2 - P_1)$ e $(P_3 - P_2)$: elas indicam os **vetores direção entre os pontos de controle**, que definem a tangente da curva em cada t .

Aplicação Prática e Normalização

Vetor de Direção: O resultado de $B'(t)$ nos dá um **vetor tangente**.

Normalização: Frequentemente, queremos apenas a direção, ignorando a velocidade. Para isso, normalizamos o vetor.

Nota Técnica: Se a magnitude da derivada for zero (ponto de singularidade ou cuspide), a **direção é indefinida**. Isso ocorre tipicamente quando os pontos de controle coincidem ou se sobrepõem de forma colinear.

Curvas Spline

Um desafio ao gerar caminhos complexos com Bézier ou Hermite é garantir que a junção entre os segmentos seja suave.

- Em **Bézier**, isso exige o **alinhamento manual** e rigoroso dos pontos de controle. Em **Hermite**, o **programador** precisaria **definir cada vetor tangente individualmente**

As Splines resolvem esse problema: a própria formulação matemática calcula as transições entre os pontos, **garantindo suavidade automaticamente**, sem a necessidade de alinhar tangentes "na mão".

Divisão da Família:

- **Aproximação (B-Splines)**: Foco em suavidade (C^2). A curva é atraída, mas não toca os pontos.
- **Interpolação (Catmull-Rom)**: Foco em precisão (C^1). A curva passa obrigatoriamente sobre os pontos.

Continuidade de Curvas (C0, C1 e C2)

Continuidade define quão suave é a transição entre segmentos de curva.

Para unir vários segmentos de curva e formar um caminho suave, precisamos garantir a **continuidade** nas junções:

Continuidade C^0 (Posição): O fim de um segmento é o início do próximo. A curva é contínua, mas pode ter "bicos" (mudanças bruscas de direção).

Continuidade C^1 (Tangente/Velocidade): A direção (vetor tangente) na junção é a mesma para os dois segmentos. O movimento é suave.

Continuidade C^2 (Curvatura/Aceleração): A taxa de mudança da tangente é a mesma. Essencial para câmeras de jogo ou pistas de corrida, evitando "trancos" na aceleração centrípeta.

Descrição matemática: B-Splines Cúbicas

A B-Spline Cúbica é uma curva de aproximação que garante continuidade C^2 em toda a sua extensão.

A curva é definida por uma janela deslizante de 4 pontos de controle simultâneos (P_0, P_1, P_2, P_3). A equação paramétrica para avaliar um segmento da curva, com t variando de 0 a 1, é:

$$S(t) = \frac{1}{6} [(1-t)^3 P_0 + (3t^3 - 6t^2 + 4) P_1 + (-3t^3 + 3t^2 + 3t + 1) P_2 + t^3 P_3]$$

Comportamento geométrico: A curva resultante não passa por **P_0** nem por **P_3** . Ela desenha um segmento central que é atraído pelos pontos, garantindo a ausência de mudanças bruscas de trajetória.

Catmull-Rom (Interpolação)

Catmull-Rom é uma variação prática da curva de Hermite. Ela é uma curva de **interpolação** (passa sobre os pontos de controle):

- **A sacada didática:** Ela usa a **mesma equação da Hermite**, mas calcula os vetores tangentes (**M1** e **M2**) automaticamente usando os pontos vizinhos.
- A curva é construída segmento por segmento. Para desenhar o trecho entre **P1** e **P2**, utilizamos também o ponto anterior (**P0**) e o próximo (**P3**).

Cálculo das Tangentes (Derivação de Hermite):

- $M1 = (P2 - P0)/2$ (A direção em P1 é a reta entre seus vizinhos).
- $M2 = (P3 - P1)/2$ (A direção em P2 é a reta entre seus vizinhos).

Substituindo essas tangentes na fórmula de Hermite, obtemos a equação do segmento entre P1 e P2, com **t** variando de 0 a 1:

$$[x,y] = 0.5 \cdot [(2P1) + (-P0 + P2)t + (2P0 - 5P1 + 4P2 - P3)t^2 + (-P0 + 3P1 - 3P2 + P3)t^3]$$

Qual fórmula usar? (O Critério de Escolha)

Você não usa as duas ao mesmo tempo. A escolha depende do objetivo do seu projeto de jogo ou ferramenta:

Tipo de Spline	Objetivo	Comportamento Geométrico	Continuidade
B-Spline	Suavidade máxima	Aproximação: A curva não passa pelos pontos de controle.	C2 (Suavíssima)
Catmull-Rom	Precisão de caminho	Interpolação: A curva obrigatoriamente passa por P1 e P2.	C1 (Suave)

Implementação (Caminhos Complexos)

Para implementar um caminho com N pontos ($P_0, P_1, \dots, P_n$):

1. **Janela Deslizante:** Pegamos grupos de 4 pontos por vez.
 - a. 1º Segmento: usa $\{P_0, P_1, P_2, P_3\}$ para desenhar de P_1 até P_2 .
 - b. 2º Segmento: usa $\{P_1, P_2, P_3, P_4\}$ para desenhar de P_2 até P_3 .
2. **Precisão:** Definimos o "pulo" de t (ex: 0.05 para 20 segmentos de reta por trecho).
3. **Loop:** Para cada valor de t , calculamos $[x,y]$ e desenhamos uma reta até o próximo ponto calculado.